Software Engineering

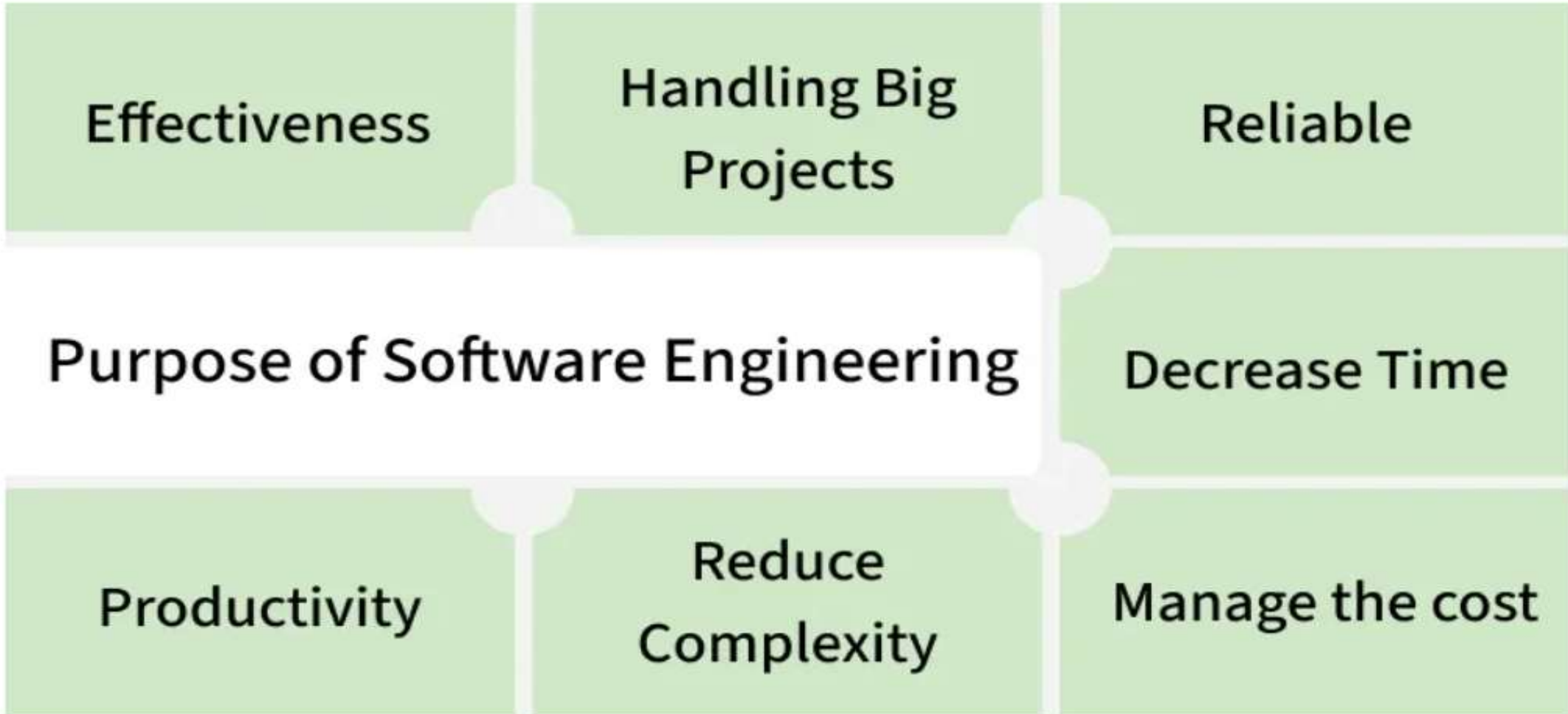
- Evaluation Scheme Theory:
- MSE: 30 Marks, TA: 20 Marks, ESE: 50
- Marks Laboratory: CIE: 50 Marks, ESE: 50 Marks

Software Engineering

- Software Engineering is the systematic process of designing, developing, testing, and maintaining software to create reliable, efficient, and high-quality applications.
- It applies engineering principles, methods, and tools to develop software in a systematic manner.
- It supports teamwork, planning, documentation, testing, and maintenance to ensure successful software development throughout its lifecycle.
- It helps create software that meets user requirements while ensuring reliability,

Need of Software Engineering

- Software engineering is a technique through which we can develop or create software for computer systems or any other electronic devices.
- It is a systematic, scientific and disciplined approach to the development, functioning, and maintenance of software.



Purpose of Software Engineering

- **Handling Big Projects**: A corporation must use a software engineering methodology in order to handle large projects without any issues.
- **To manage the cost**: Software engineering programmers plan everything and reduce all those things that are not required.
- **To decrease time**: It will save a lot of time if you are developing software using a software engineering technique.
- **Reliable software**: It is the company's responsibility to deliver software products on schedule and to address any defects that may

- **Effectiveness**: Effectiveness results from things being created in accordance with the standards.
- **Reduces complexity**: Large challenges are broken down into smaller ones and solved one at a time in software engineering. Individual solutions are found for each of these issues.
- **Productivity**: Because it contains testing systems at every level, proper care is done to maintain software productivity.

Software Crisis and Myths,

- **What is Software Crisis?**
- **Software Crisis** is a term used in computer science for the difficulty of writing useful and efficient computer programs in the required time.
- The software crisis was due to using the same workforce, same methods, and same tools even though rapidly increasing software demand, the complexity of software, and software challenges.

- Suppose we use the same workforce, same methods, and same tools after the fast increase in software demand, software complexity, and software challenges.
- In that case, there arise some issues like software budget problems, software efficiency problems, software quality problems, software management, and delivery problems, etc. This

Causes of Software Crisis

- The cost of owning and maintaining software was as expensive as developing the software.
- At that time Projects were running overtime.
- At that time Software was very inefficient.
- The quality of the software was low quality.
- Software often did not meet user requirements.
- At that time Software was never delivered

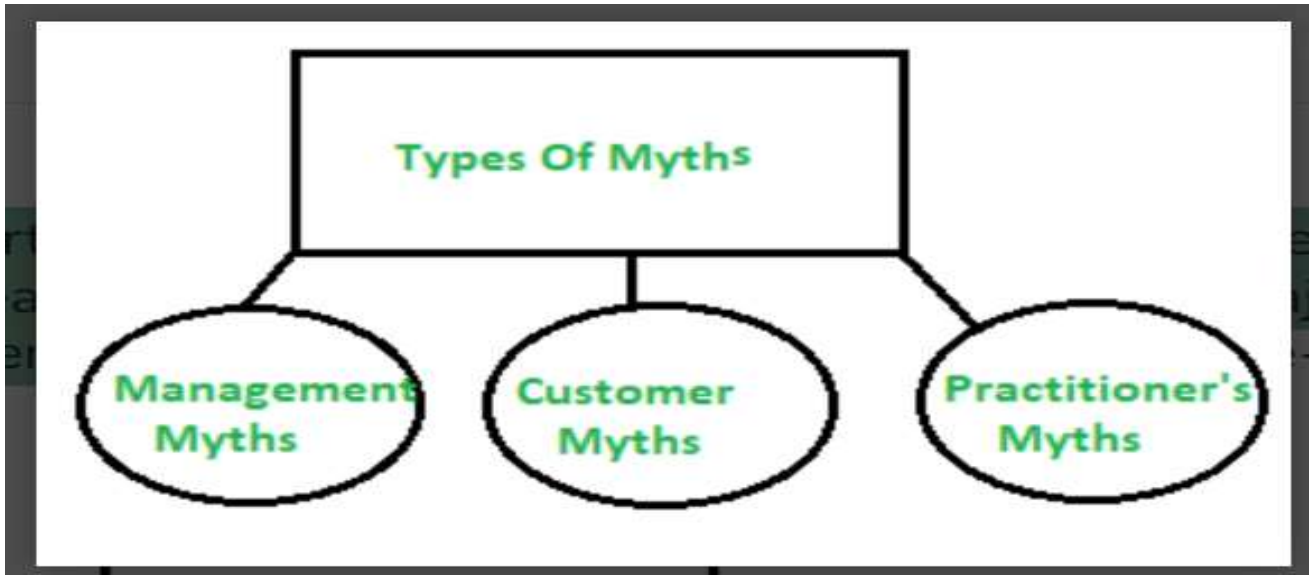
- **Factors Contributing to Software Crisis**
- Factor Contributing to Software Crisis are:
- Poor project management.
- Lack of adequate training in software engineering.
- Less skilled project members.
- Low productivity improvements.

- **Solution of Software Crisis:**
- There is no single solution to the crisis. One possible solution to a software crisis is *Software Engineering* because software engineering is a systematic, disciplined, and quantifiable approach. For preventing software crises, there are some guidelines:

- Reduction in software over budget.
- The quality of the software must be high.
- Less time is needed for a software project.
- Experienced and skilled people working on the software project.
- Software must be delivered.
- Software must meet user requirements.

Software Myths

- Most, experienced experts have seen myths or superstitions (false beliefs or interpretations) or misleading attitudes (naked users) which creates major problems for management and technical people(leading to poor planning, communication issues, increased costs, delays, and lower software quality.


Types Of Myths
Management Myths
Customer Myths
Practitioner's Myths

(i) Management Myths:

- **Myth 1:**
- We have all the standards and procedures available for software development.
- Management may believe that **having standards, guidelines, and procedures alone is enough to ensure a successful software project.**

Fact:-

- Standards and procedures provide only a **framework** for development

- Success also depends on:
  - Skilled and experienced developers
  - Proper project planning
  - Good communication among team members
  - Effective project management
  - Regular testing and quality assurance
  - Continuous monitoring and risk management

- **Myth 2:**
- The addition of the latest hardware programs will improve the software development.
- Fact:-
- New hardware and tools **can help**, but they **do not guarantee** better software.
- The success of software development depends mainly on:
    - Skilled and experienced developers
    - Well-defined requirements
    - Proper planning and project management
    - Effective communication
    - Good software engineering practices

- **Myth 3:**
- With the addition of more people and program planners to Software development can help meet project deadlines (If lagging behind)
- Fact:-
- New team members need time to understand the project.
- Existing developers must spend time training and coordinating with them.
- Communication and coordination become more complex as the team grows. As a result, the project may become **even more delayed.**

- **(ii)Customer Myths:**
- The customer can be the direct users of the software, the technical team, marketing / sales department, or other company.
- **Myth 1:**
- A general statement of intent is enough to start writing plans (software development) and details of objectives can be done over time.
- Fact:-
- Software development requires **clear, complete, and detailed requirements** before starting.

- If requirements are incomplete or keep changing:

  Developers may misunderstand customer needs.

  More time and effort are required to make changes.

  Development cost increases.

  The project may be delayed.

  Software quality may be affected.

- **Myth 2:**
- Software requirements continually change, but change can be easily accommodated because software is flexible.
- Fact:-
- Although software is flexible, **every change affects the project.**
- Requirement changes may require:

  Modifying the design

  Changing the code

  Updating documentation

  Retesting the software

- As development progresses, the **cost and effort of making changes increase**
- Frequent changes can cause:
  - Project delays
  - Increased development cost
  - More defects (bugs)
  - Reduced software quality

- **iii) Practitioner's Myths:**
- **Myths 1:**
- They believe that their work has been completed with the writing of the plan.
- Fact:-
- software development does not end with coding. After coding, several important activities are still required:
- Testing the software to find and fix bugs.

- Preparing documentation.
- Integrating the software with other modules.
- Deploying the software.
- Maintaining and updating the software after delivery.

- Myths 2:

There is no other way to achieve system quality, until it is "running".

Fact:-

- software quality can be ensured **throughout the development process,** not just after execution.
- Quality can be improved by:
- Requirement reviews

- Design reviews
- Code inspections
- Unit testing
- Integration testing
- Static analysis
- Verification and validation

- **Myth 3:**
- An operating system is the only product that can be successfully exported project.
- Some developers believe that **once the software runs correctly, the project is complete and nothing else needs to be delivered.**

- **Fact:-**
- successful software project includes many deliverables besides the working software, such as:
- Software documentation
- Design documents
- Test cases and test reports
- User manuals
- Installation and maintenance guides
- Training materials (if required)

- **Myth 4:**
- Engineering software will enable us to build powerful and unnecessary document & always delay us.
- Documentation is an important part of software engineering. Good documentation helps in:

    Understanding the system

    Team communication

    Software maintenance

    Future enhancements

- Software engineering promotes **necessary and useful documentation**, not unnecessary paperwork.

Software Quality Attributes,

- Software Quality Attributes are the characteristics that determine how well a software system performs and satisfies user requirements. They define the quality, reliability, efficiency, and usability of software.

- Definition
- Software Quality Attributes are measurable properties that describe the quality of a software system and determine whether it meets

- Major Software Quality Attributes

1. Correctness

- The software performs the required functions correctly.
- It satisfies all specified user requirements.

2. Reliability

- The software works consistently without failures.
- It performs accurately under specified conditions for a given period.

3. Efficiency

- The software uses system resources (CPU, memory, storage, network) effectively.
- It provides fast response and good performance.

4. Integrity (Security)

- Protects data from unauthorized access or modification.
- Ensures confidentiality and data security

5. Usability

- The software is easy to learn, understand, and use.
- It provides a user-friendly interface

6. Maintainability

- The software can be easily modified, corrected, or upgraded.
- Makes fixing bugs and adding new features easier.

7. Flexibility

- The software can be adapted to changing user requirements.
- New features can be added with minimal effort.

8. Testability

- The software can be easily tested to detect defects.
- Supports effective verification and validation.

9. Portability

- The software can run on different operating systems or hardware platforms with little modification.

10. Reusability

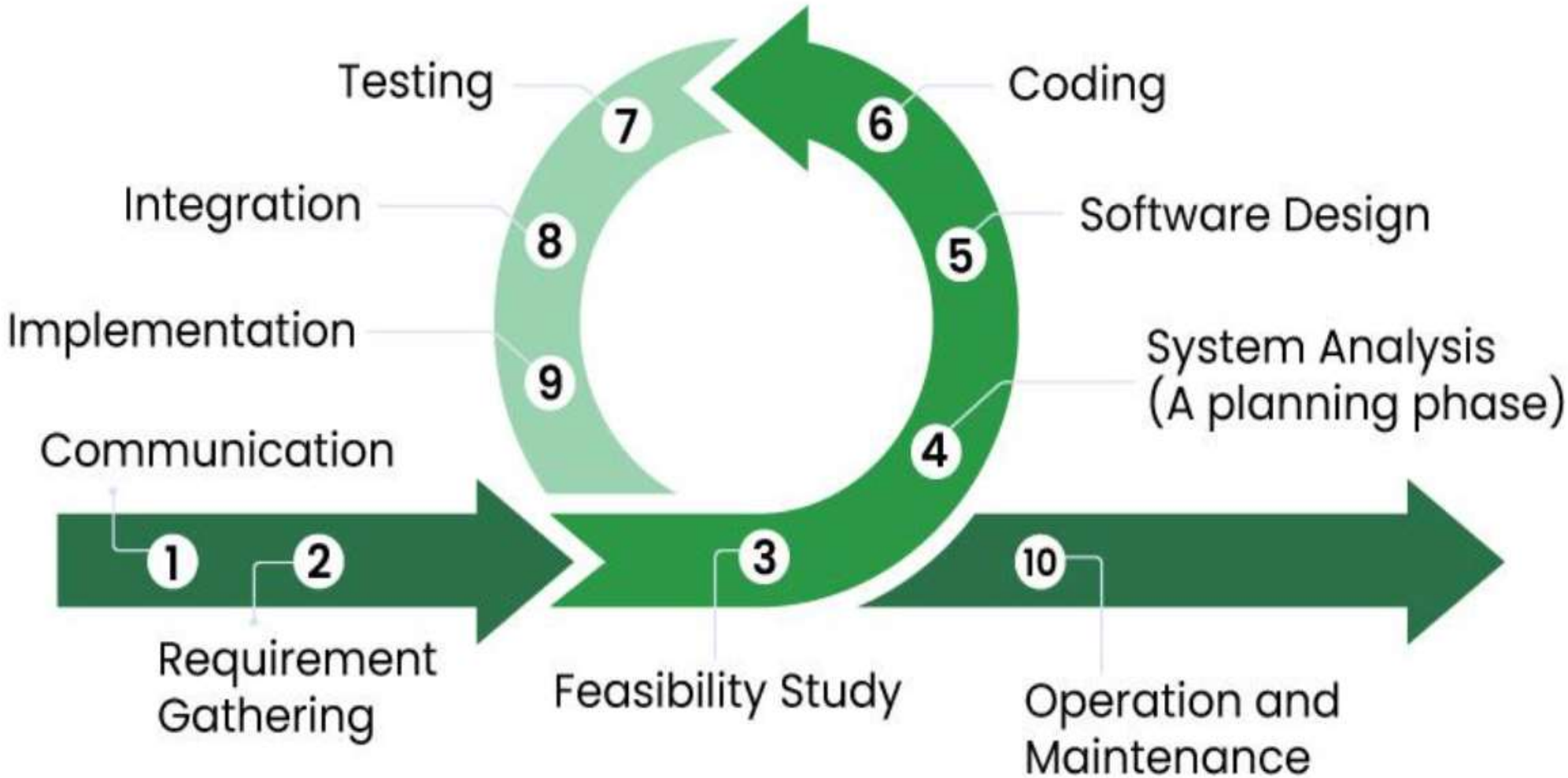
- Software components can be reused in other applications.
- Reduces development time and cost.

11. Interoperability

- The software can communicate and exchange data with other systems.

Software Process and development

- **What is Software Development Process?**
- The software development process is the approach to developing, and delivering software applications. This process might include improving design and product management by splitting the work into smaller steps or processes.



Software development process

1. Communication

- Developers meet with the client or stakeholders.
- They discuss the problem and understand what the software should do.
- **Goal:** Understand the client's needs.

2. Requirement Gathering

- Collect all functional and non-functional requirements.
- Document them clearly.
- **Output:** Software Requirements Specification (SRS).

3. Feasibility Study

- Analyse whether the project is possible.
- Check:
  - **Technical feasibility**: Can it be built?
  - **Economic feasibility**: Is it within budget?
  - **Operational feasibility**: Will users accept it?
  - **Time feasibility**: Can it be completed on time?
- **Goal**: Decide whether to proceed.

4. System Analysis (Planning Phase)

- Study the requirements in detail.
- Break the project into modules.
- Plan resources, schedule, risks, and architecture.
- **Goal**: Create a clear development plan.

5. Software Design

- Design how the software will work.
- Create:
  - Database design
  - User interface (UI)
  - System architecture
  - Algorithms
- **Output:** Design documents.

6. Coding

- Developers write the actual program using programming languages such as Java, Python, C++, or JavaScript.
- **Goal**: Convert the design into working software.

7. Testing

- Testing is the process of checking whether the software works correctly and meets the user's requirements.
- It helps identify and fix errors (bugs) before the software is delivered to users.

- Common testing types:
  - Unit Testing
  - Integration Testing
  - System Testing

8. Integration

Integration is the process of combining all the individual software modules (frontend, backend, database, APIs, and libraries) into one complete system so they work together correctly.

9. Implementation (Deployment)

Implementation is the stage where the completed software is installed and made available for users to use in the real environment.

10. Operation and Maintenance

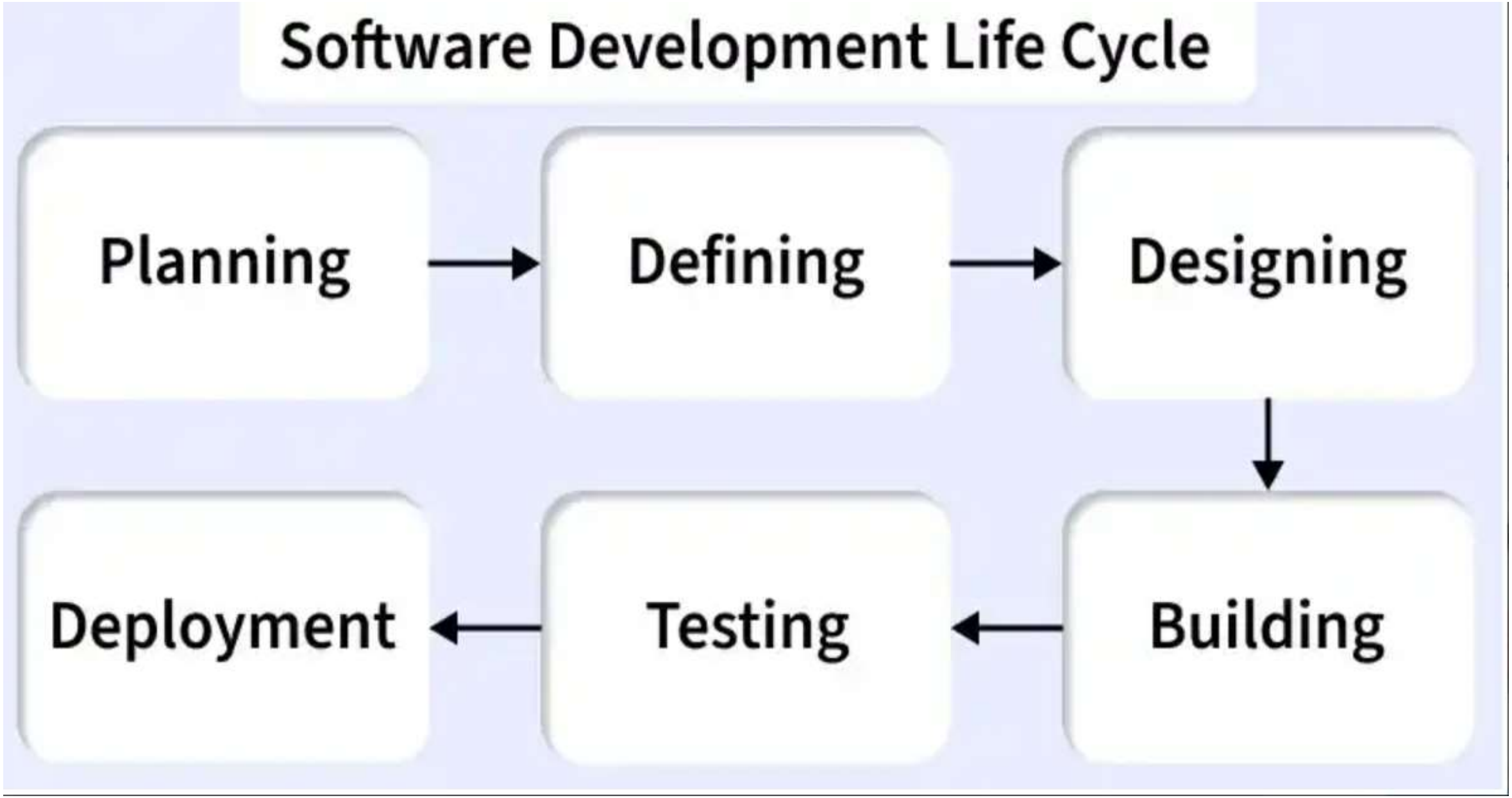
Operation and Maintenance is the final phase of the Software Development Process (SDP). In this phase, the software is used by end users, monitored for performance, and regularly updated to fix bugs, improve features, and adapt to changing technologies or user requirements.

- **Activities in Operation and Maintenance**
- Monitor software performance.
- Fix bugs and errors.
- Provide user support and documentation.
- Update the software with new features.
- Improve performance and security.
- Ensure compatibility with new technologies and operating systems.

Software life cycle and Models

- The **Software Development Life Cycle (SDLC)** is a structured process used to **plan, design, develop, test, deploy, and maintain software.** It provides a systematic approach to building software that meets business goals and user requirements.

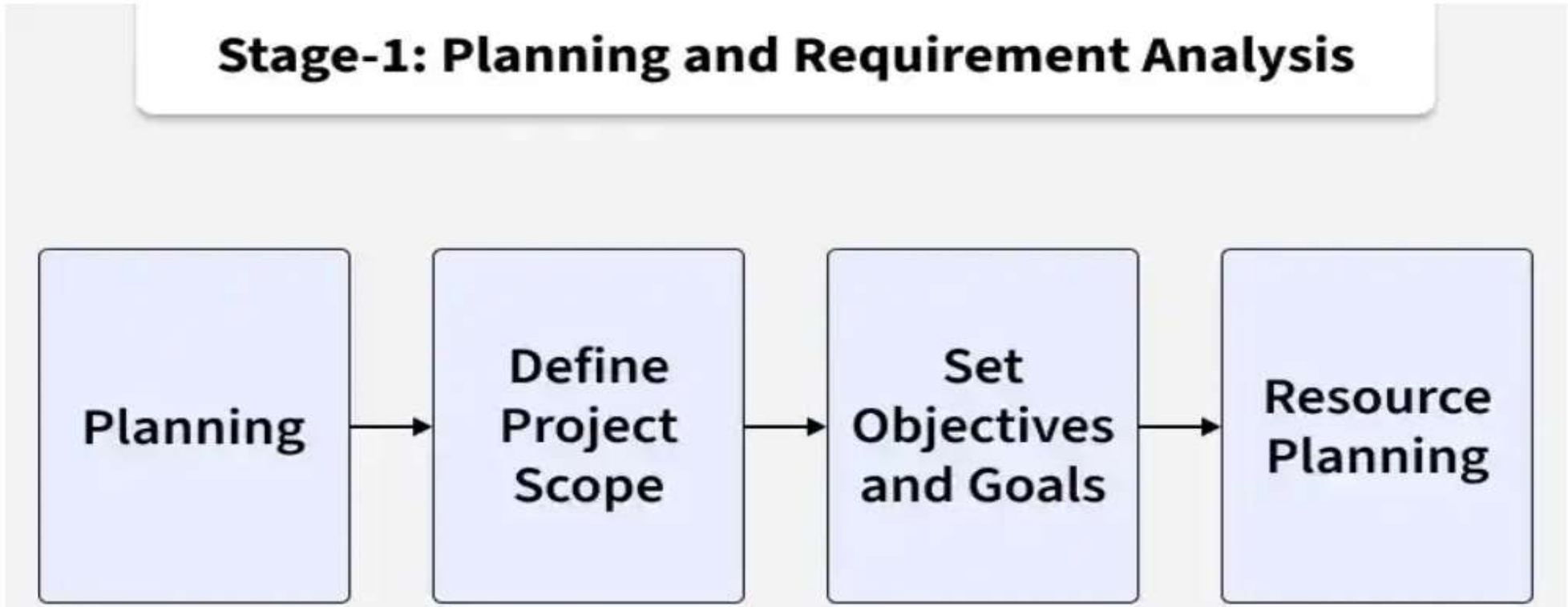
- **Definition**
- **Software Development Life Cycle (SDLC)** is a step-by-step process followed by software developers to create high-quality software efficiently and systematically


Software Development Life Cycle
Planning
Defining
Designing
Building
Testing
Deployment

Stage 1: Planning & Feasibility Analysis

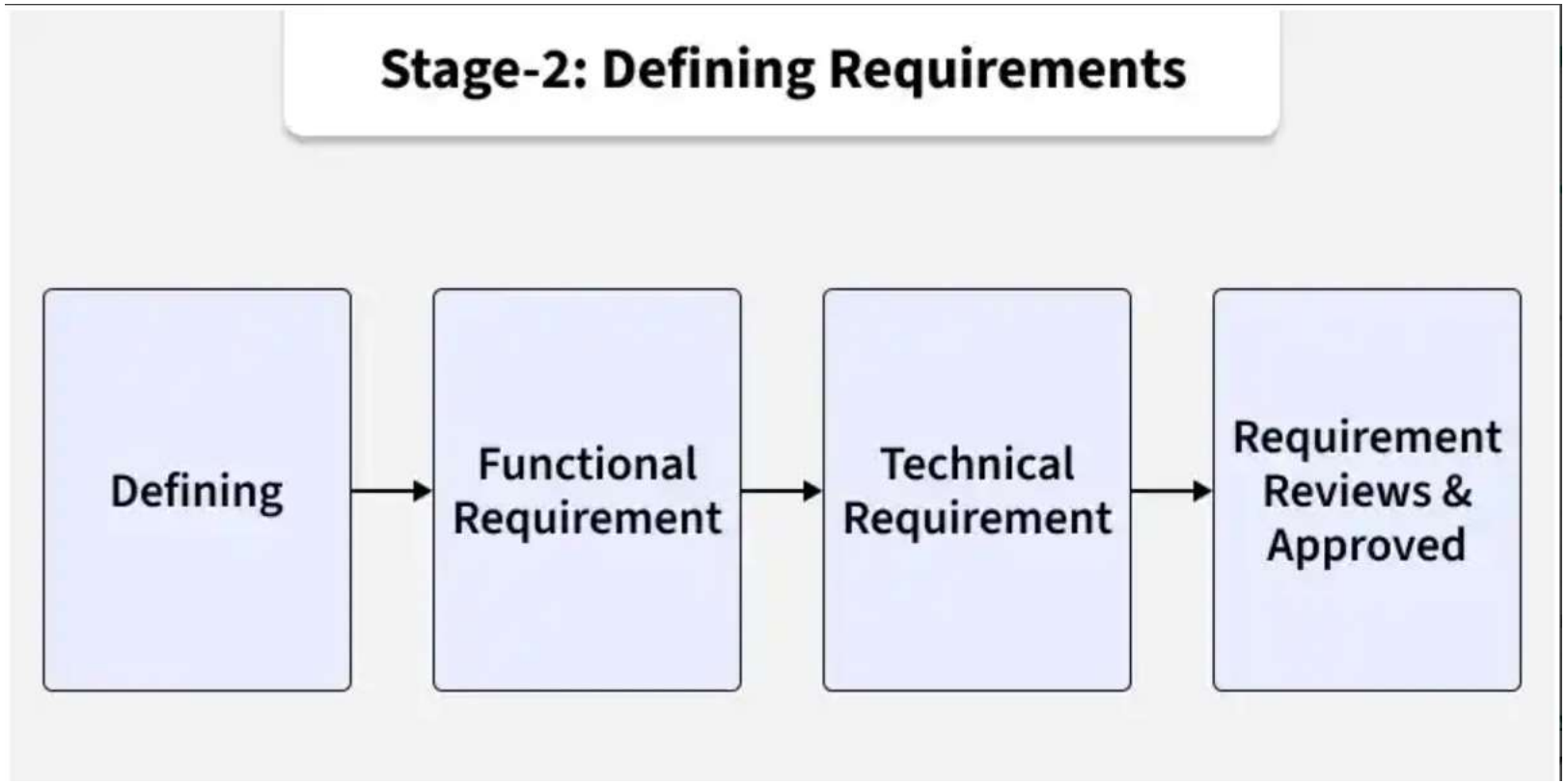
- This is the **first stage of the Software Development Life Cycle (SDLC).** In this stage, the organization decides whether the proposed software project is practical and worth developing.

- **Definition**
- **Planning and Feasibility Analysis** is the process of identifying project goals, estimating resources, cost, time, and analyzing whether the project is **technically, financially, and operationally feasible.**


Stage-1: Planning and Requirement Analysis
Planning
Define Project Scope
Set Objectives and Goals
Resource Planning

- **Stage 2: Requirement Specification (SRS)**
- This is the **second stage of the Software Development Life Cycle (SDLC).** In this stage, all the software requirements are collected, analyzed, documented, and approved by the stakeholders.

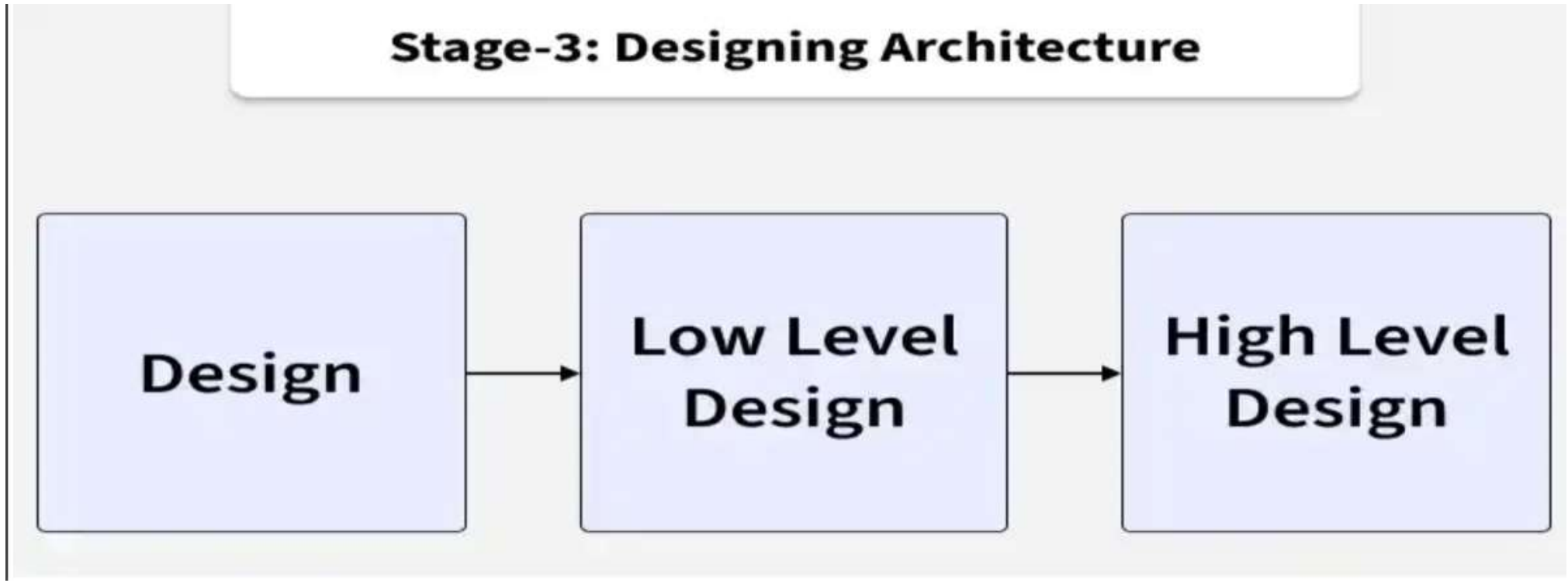
- **Definition**
- **Requirement Specification (SRS)** is the process of identifying, documenting, and validating all **functional and non-functional requirements** of the software before development begins.


Stage-2: Defining Requirements
Defining
Functional Requirement
Technical Requirement
Requirement Reviews & Approved

Stage 3: System Design

- The **System Design** phase is the **third stage of the Software Development Life Cycle (SDLC)**. In this stage, the approved requirements from the **SRS document** are converted into a **technical blueprint** for software development.

- Definition:-
- **System Design** is the process of creating the architecture and detailed design of the software based on the approved requirements. It defines how the system will be built.


Stage-3: Designing Architecture
Design
Low Level Design
High Level Design

1. High-Level Design (HLD)

- High-Level Design describes the overall structure of the system.
- **It includes:**
- System architecture
- Technology stack (programming languages, frameworks, tools)
- Database design
- Major modules and their interactions

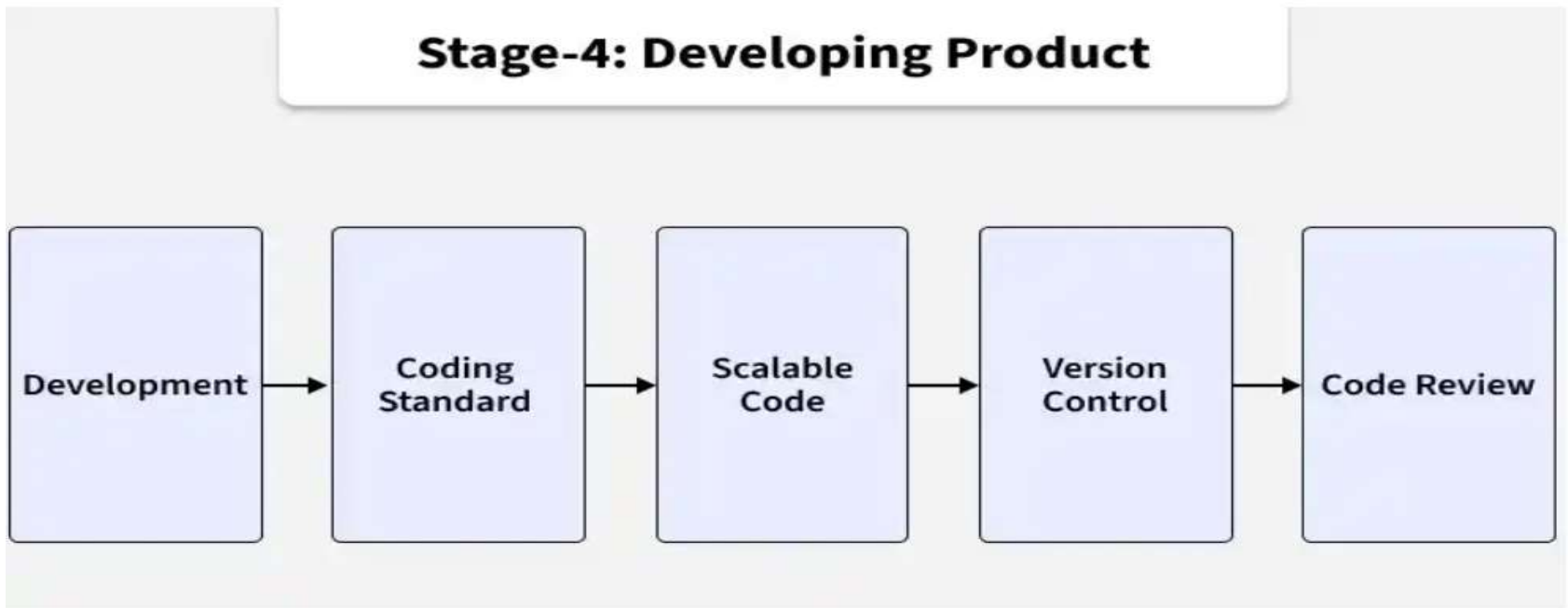
2. Low-Level Design (LLD)

- Low-Level Design provides detailed information about each module.
- **It includes:**
- Component logic
- APIs (Application Programming Interfaces)
- Data structures
- Algorithms
- Workflows and process flow
- Class and sequence diagrams

Stage 4: Development (Coding)

- The **Development (Coding)** phase is the **fourth stage of the Software Development Life Cycle (SDLC)**. In this stage, developers write the actual program code based on the **Design Document Specification (DDS)** created during the design phase.

- **Definition**
- **Development (Coding)** is the process of converting the software design into executable source code using programming languages and development tools.


Stage-4: Developing Product
Development
Coding Standard
Scalable Code
Version Control
Code Review

Stage 5: Testing

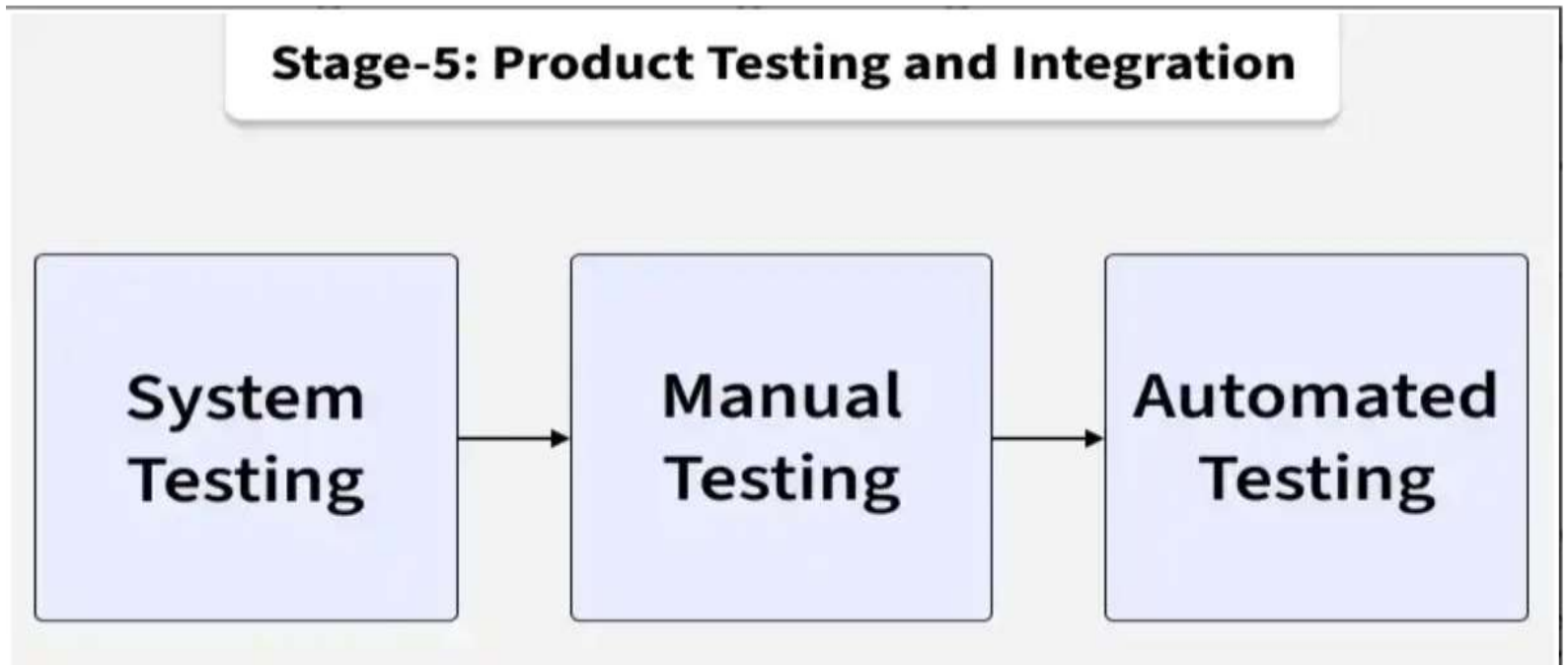
- The **Testing** phase is the **fifth stage of the Software Development Life Cycle (SDLC).** In this stage, the software is tested to ensure it **meets the specified requirements and is free from defects (bugs)** before being released to users.

Definition

- **Testing** is the process of verifying and validating the software to ensure it works correctly, meets user requirements, and

Types of Testing include:

- **Unit Testing:** Verifies individual components
- **Integration Testing:** Ensures modules work together
- **System Testing:** Validates the complete system
- **User Acceptance Testing (UAT):** Confirms business requirements are met.


Stage-5: Product Testing and Integration
System Testing
Manual Testing
Automated Testing

Stage 6: Deployment

- The **Deployment** phase is the **sixth stage of the Software Development Life Cycle (SDLC).** In this stage, the **tested software is released to the production environment** so that end users can use it.

- **Definition**
- **Deployment** is the process of installing, configuring, and releasing the software into the production environment for users.

Stage 7: Maintenance

- The **Maintenance** phase is the **final stage of the Software Development Life Cycle (SDLC)**. After the software is deployed, it is continuously monitored, updated, and improved to ensure it remains reliable, secure, and useful for users.

- **Importance of SDLC**

- Provides a clear and organized development framework.
- Improves planning, cost control, and project management.
- Ensures better quality through defined testing phases.
- Helps deliver software that meets user and business needs

Example: Online Library Management System

- **1. Planning & Feasibility Analysis**
- The college decides to develop an **Online Library Management System.**
- The team checks whether the project is technically, financially, and operationally feasible.
- **Output:** Project plan and feasibility report.

2. Requirement Specification (SRS)

- Requirements are collected from the librarian, students, and college management.
- **Functional Requirements:**
  - User login
  - Search books
  - Issue and return books
  - Fine calculation
- **Non-Functional Requirements:**
  - Fast response time
  - Secure login
  - Easy-to-use interface

3. System Design

- The system architecture, database, and modules are designed.
- **HLD:** User Management, Book Management, Issue/Return, Reports.
- **LLD:** Login logic, database tables, APIs, workflows.

4. Development (Coding)

- Developers write the source code.
- Frontend developers create the user interface.
- Backend developers develop the database and APIs.

5. Testing

- The software is tested for defects.
- **Unit Testing**: Test the login module.
- **Integration Testing**: Check communication between modules.
- **System Testing**: Test the complete library system.
- **UAT**: Librarians and students verify the software.

6. Deployment

- The application is installed on the college server.
- Smoke testing ensures the main features work correctly.

7. Maintenance

- Bugs are fixed after deployment.
- New features such as **online book reservation** are added.
- Security updates and performance improvements are provided.

Common Models

- Waterfall Model
- Agile Model
- V-Model
- Spiral Model
- Incremental Model
- RAD Model